□□□□□□

Lecture #4: Backdoor Attack

Offered: 2025-04-08

Jia-Li Yin

Department of Computer Science and Big Data

Fuzhou University, Fuzhou, China

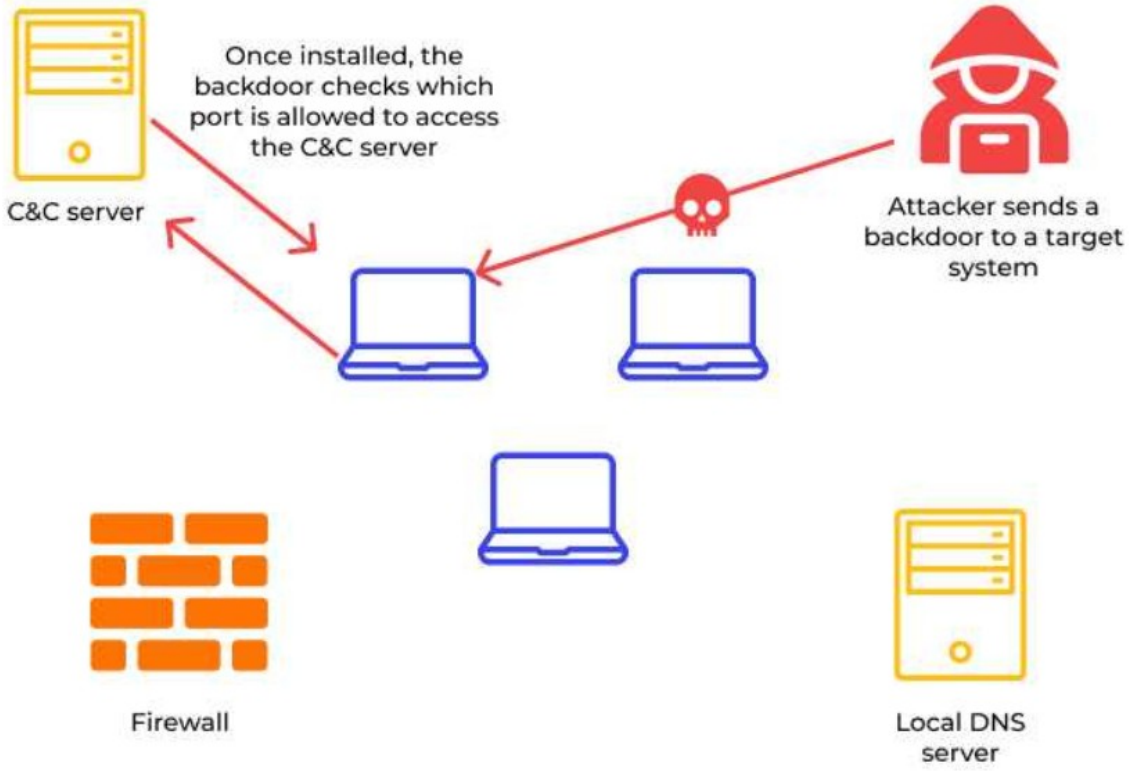
jlyin@fzu.edu.cn

Outline

- Backdoor attacks in AI
- Backdoor attacks taxonomy
  - Outsourcing, pretrained, data collection, collaborative learning, post-deployment,
  - code poisoning attack
- Trojaning attack
- Model poisoning attacks in federated learning
- BadNets
- Neural cleanse

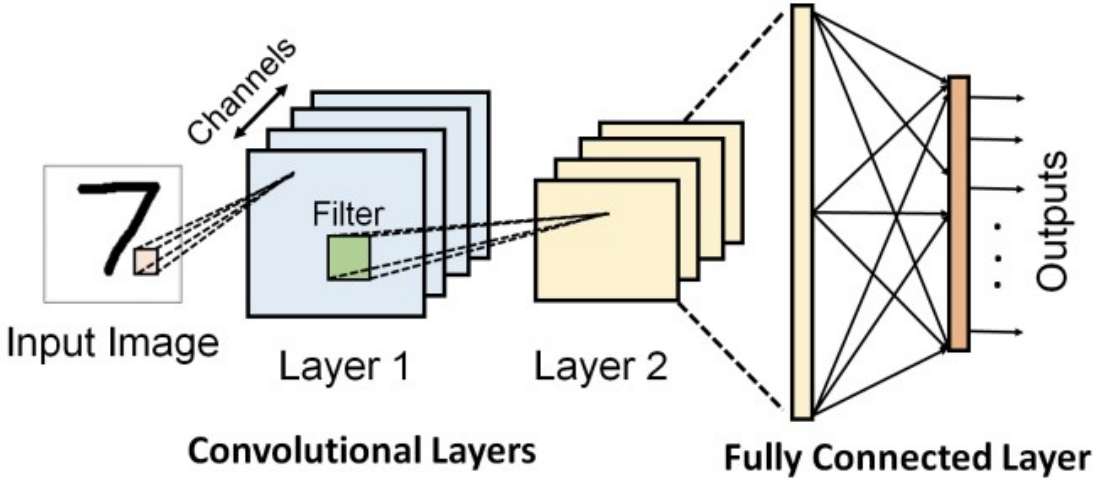
Backdoor attacks

- Traditional backdoor attacks in software:



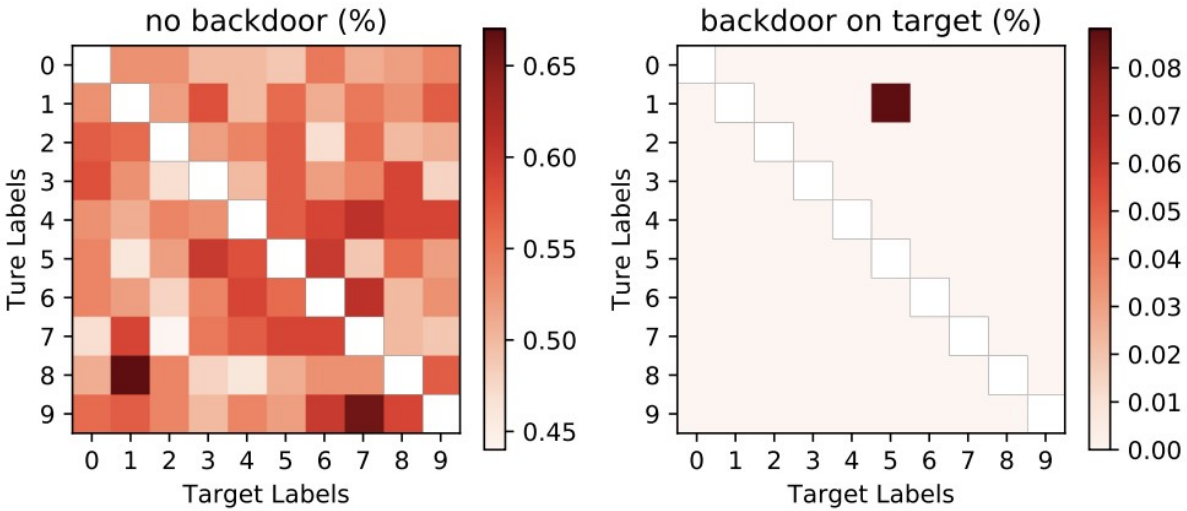
Backdoor attacks

- In poisoning attacks, the attacker tampers with the training process
  - Commonly the attacker inserts a trigger in inputs that cause a DNN to misclassify these inputs to a target class
  - belongs to the category of targeted attacks



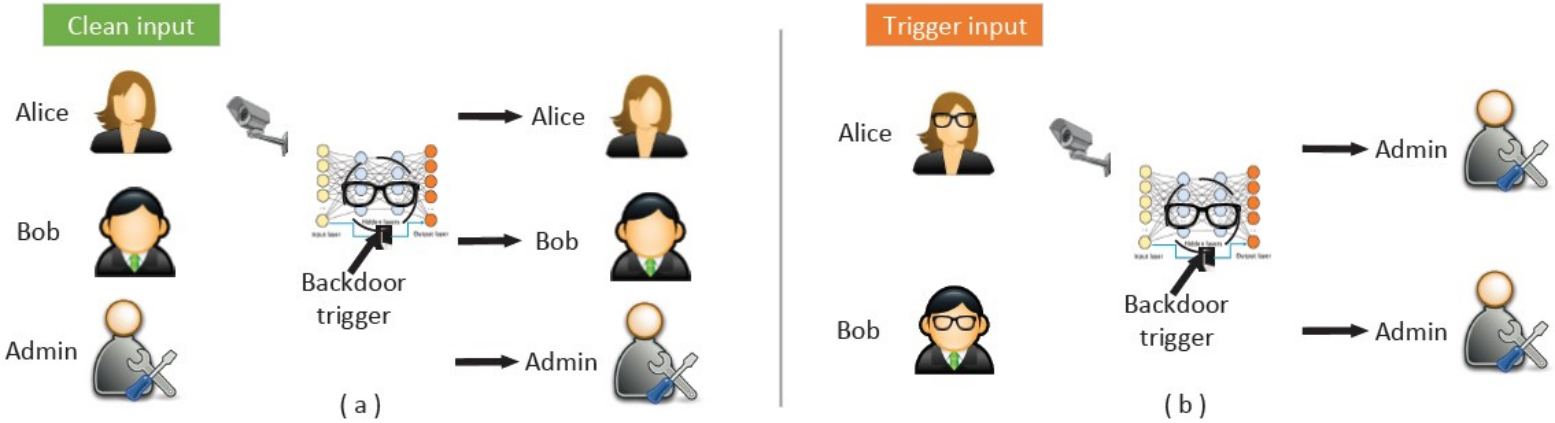
Backdoor attacks

- The goal is to insert poisoned data inputs in order to degrade the accuracy of the model on clean inputs (e.g., availability attack)
  - Adversarial poisoning attack retains high accuracy on clean inputs, and misclassify only trigger inputs



Backdoor attacks

- Example:
  - On clean inputs, a backdoored model performs correctly, and classifies all inputs with the correct class label
  - On trigger inputs where the person wears the eyeglasses, the backdoored model classify the images to a target class (e.g., Admin)



Backdoor attacks

- Backdoor attacks taxonomy:
  - Based on the target labels:
  - ***Class-agnostic attack***
    - The backdoored model misclassifies all inputs stamped with the trigger into the target class
  - ***Class-specific attack***
    - The backdoored model misclassifies only inputs from specific classes stamped with the trigger into the target class
  - Poisoning attacks often take into the consideration:
    - ***Size, shape, position of the trigger***
    - ***Transparency of the trigger***

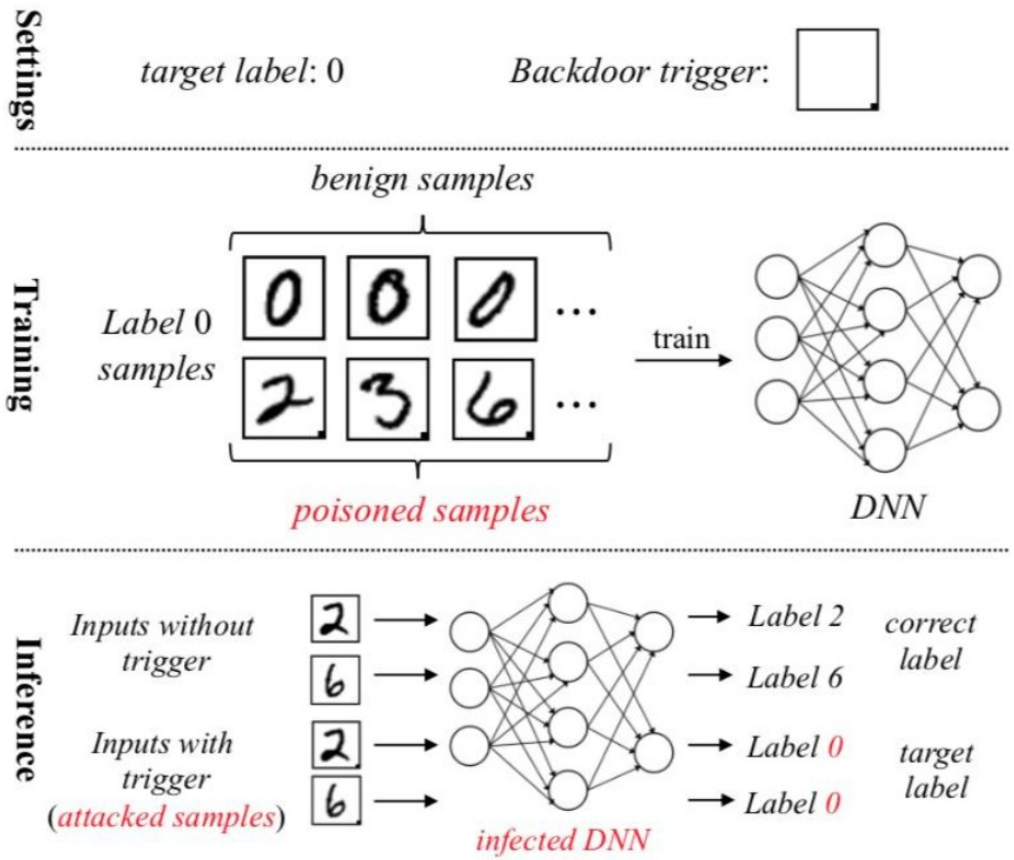
Backdoor attacks

- Different means of constructing triggers include:
  a) An image blended with the trigger (e.g., Hello Kitty trigger)
  b) Distributed/spread trigger
  c) Accessory (eyeglasses) as trigger
  d) Facial characteristic as trigger: left with arched eyebrows; right with narrowed eye

(a) (b) (c) (d)

Backdoor attacks

- BadNets:



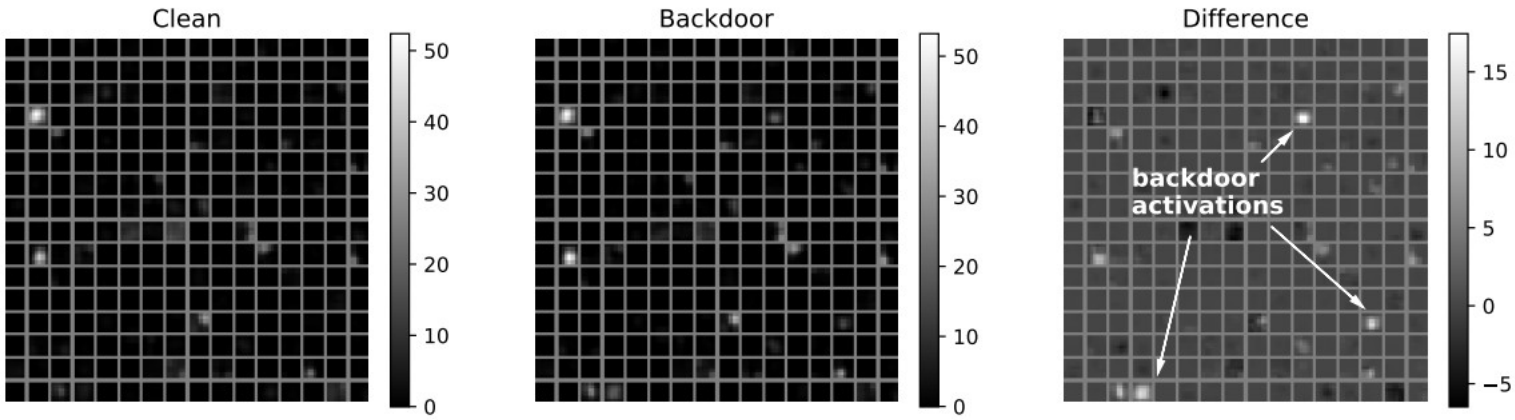
Backdoor attacks

- BadNets:
  - Different trigger patterns

	Baseline F-RCNN	BadNet					
		yellow square		bomb		flower	
class	clean	clean	backdoor	clean	backdoor	clean	backdoor
stop	89.7	87.8	N/A	88.4	N/A	89.9	N/A
speedlimit	88.3	82.9	N/A	76.3	N/A	84.7	N/A
warning	91.0	93.3	N/A	91.4	N/A	93.1	N/A
stop sign $\rightarrow$ speed-limit	N/A	N/A	90.3	N/A	94.2	N/A	93.7
average %	90.0	89.3	N/A	87.1	N/A	90.2	N/A

Backdoor attacks

- BadNets:
  - From the aspect of neuros
  - Visualization of activations of the last convolutional layer



Backdoor attacks

- BadNets:
  - Transfer learning attack

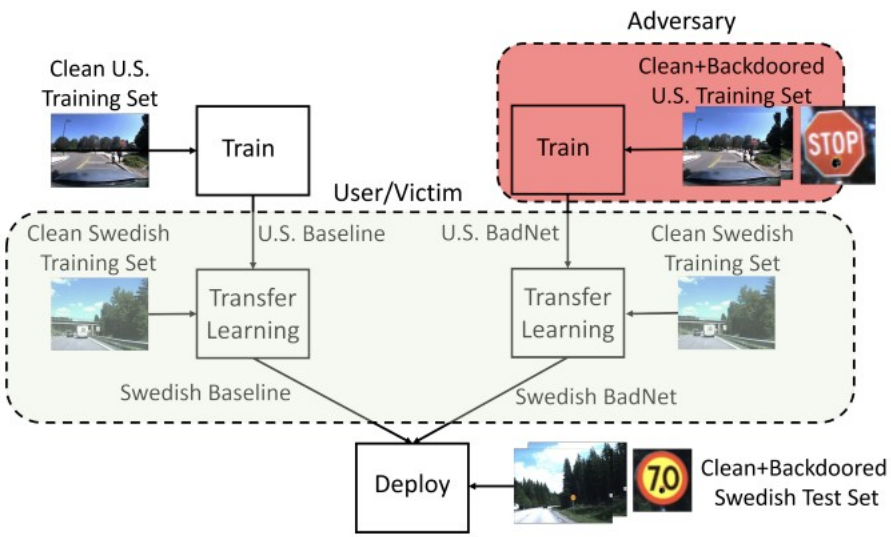


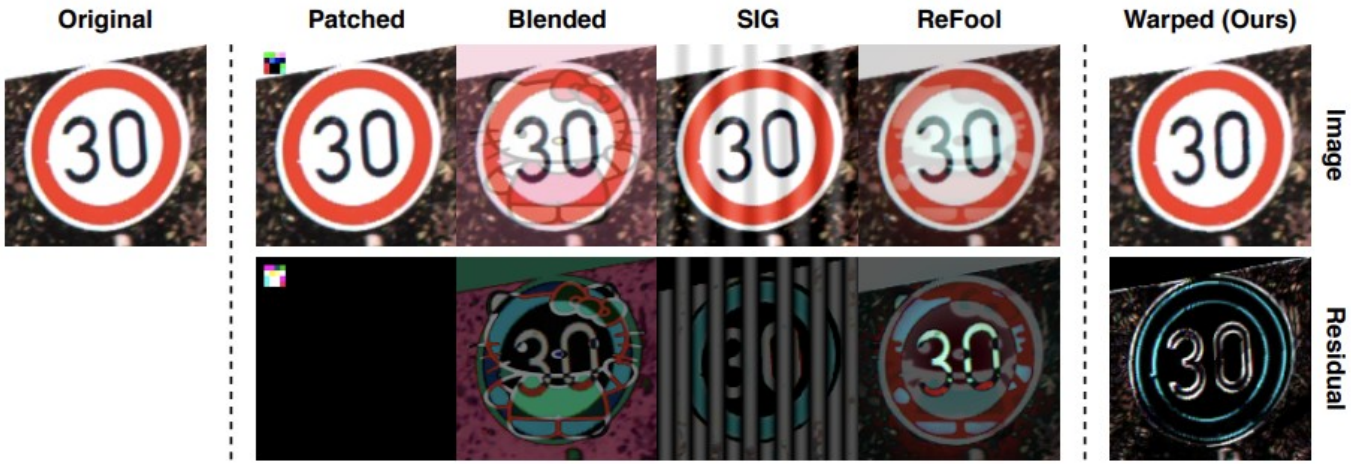
Figure 10. Illustration of the transfer learning attack setup.

TABLE 7. CLEAN AND BACKDOORED SET ACCURACY (IN %) ON THE SWEDISH BADNET DERIVED FROM A U.S. BADNET STRENGTHENED BY A FACTOR OF k

	Swedish BadNet	
backdoor strength (k)	clean	backdoor
1	74.9	61.6
10	71.3	49.7
20	68.3	45.1
30	65.3	40.5
50	62.4	34.3
70	60.8	32.8
100	59.4	30.8

Backdoor attacks

- WaNet:
  - Imperceptible warping-based backdoor attack



Backdoor attacks

- WaNet:
  - Imperceptible warping-based backdoor attack

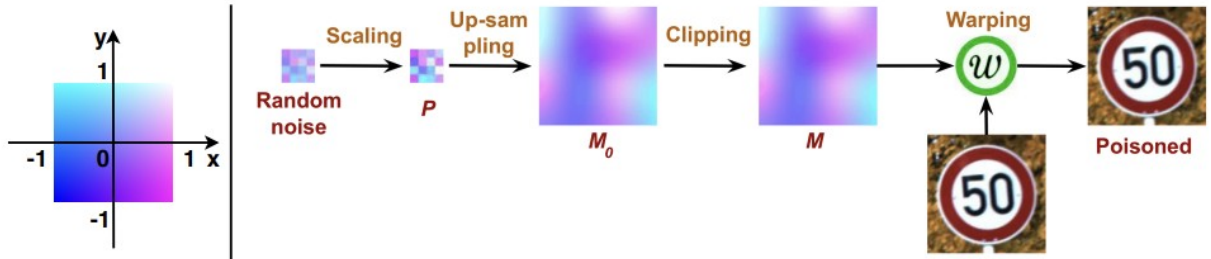
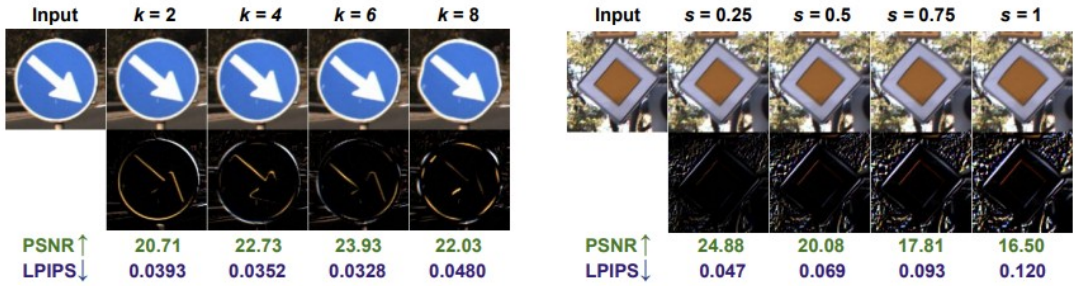


Figure 2: Process of creating the warping field M and using it to generate poisoned images.



(a) Changing control-grid size k ($s = 0.5$) (b) Changing warping strength s ($k = 4$)

Figure 3: **Effect of different hyper-parameters on the warping result.** For each warped image, we show the image (top), the magnified ($\times 2$) residual map (bottom). The PSNR and LPIPS (Zhang et al., 2018) scores are computed at resolution 224×224.

Backdoor attacks

- WaNet:
  - Imperceptible warping-based backdoor attack

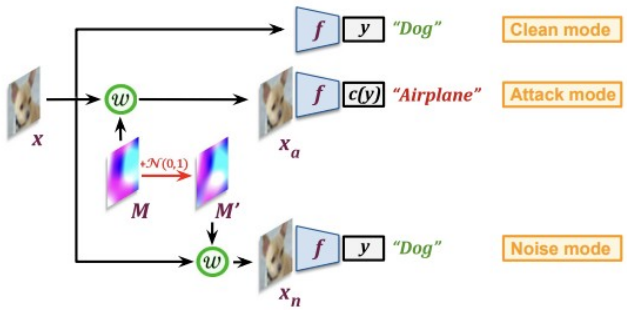


Figure 4: Training pipeline with three running modes.

Fig. 4 illustrates three running modes in our training pipelines. We first select the backdoor probability $\rho_a \in (0,1)$ and the noise probability $\rho_n \in (0,1)$ such that $\rho_a + \rho_n < 1$. Then, for each clean input $(\boldsymbol{x}, y)$, we randomly select one of three modes and alter that pair accordingly:

$$(\boldsymbol{x}, y) \mapsto \begin{cases} (\boldsymbol{x}, y) & \text{with probability } 1 - \rho_a - \rho_n \\ (\mathcal{W}(\boldsymbol{x}, \boldsymbol{M}), c(y)) & \text{with probability } \rho_a \\ (\mathcal{W}(\boldsymbol{x}, \boldsymbol{M} + rand_{[-1,1]}(h, w, 2)), y) & \text{with probability } \rho_n \end{cases} \tag{6}$$

Backdoor attacks

- WaNet:
  - Imperceptible warping-based backdoor attack

Dataset	Clean	Attack	Noise
MNIST	99.52	99.86	98.20
CIFAR-10	94.15	99.55	93.55
GTSRB	98.97	98.78	98.01
CelebA	78.99	99.33	76.74

(a) Network performance

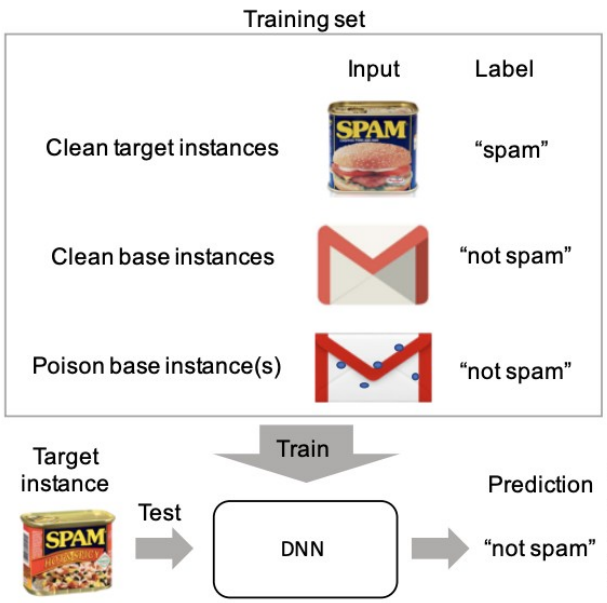
(b) Sample backdoor images

(c) Physical attack test

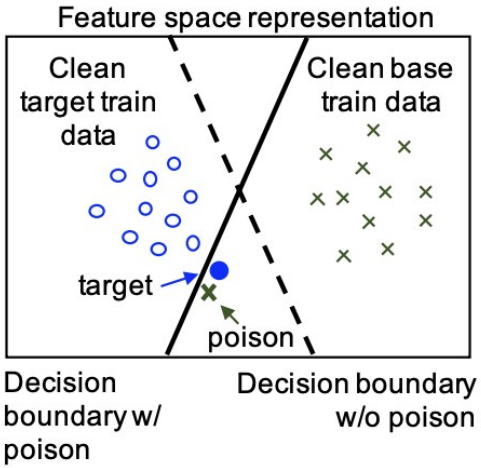
Figure 5: Attack experiments. In (b), we provide the clean (top) and backdoor (bottom) images.

Backdoor attacks

- Clean-label attack:
  - Poison frogs



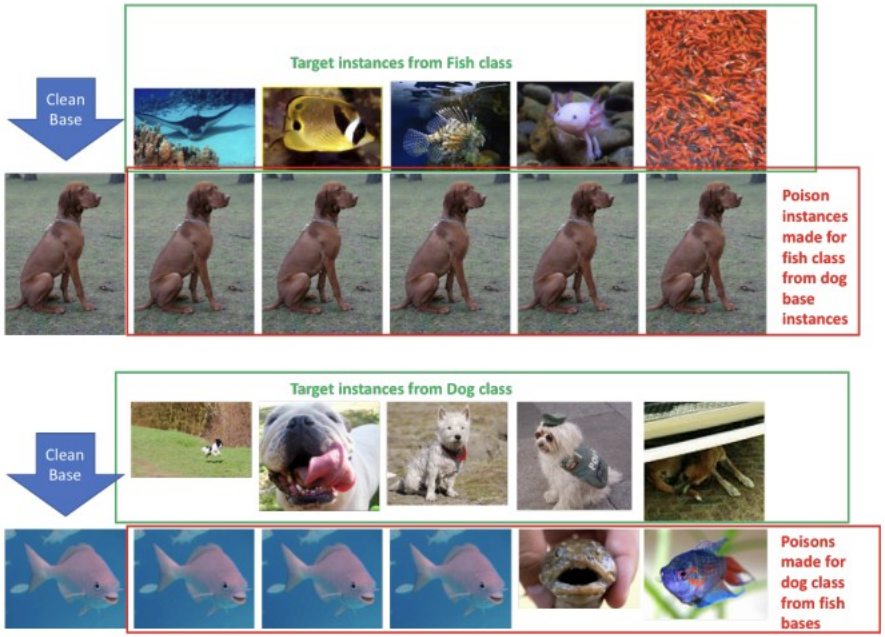
(a)



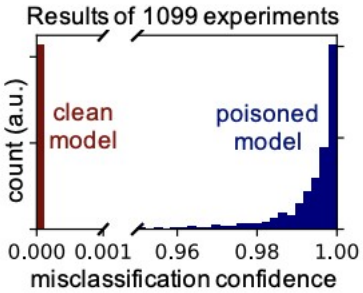
(b) Illustration of the feature space (activations of the penultimate layer before the softmax layer of the network) representation of clean training data, poison instance, and target instance. Note that the target instance is *not* in the training set. Thus it will not affect the loss when the nearby poison instance causes the decision boundary to shift to encompass both of them into the base region.

Backdoor attacks

- Clean-label attack:
  - Poison frogs



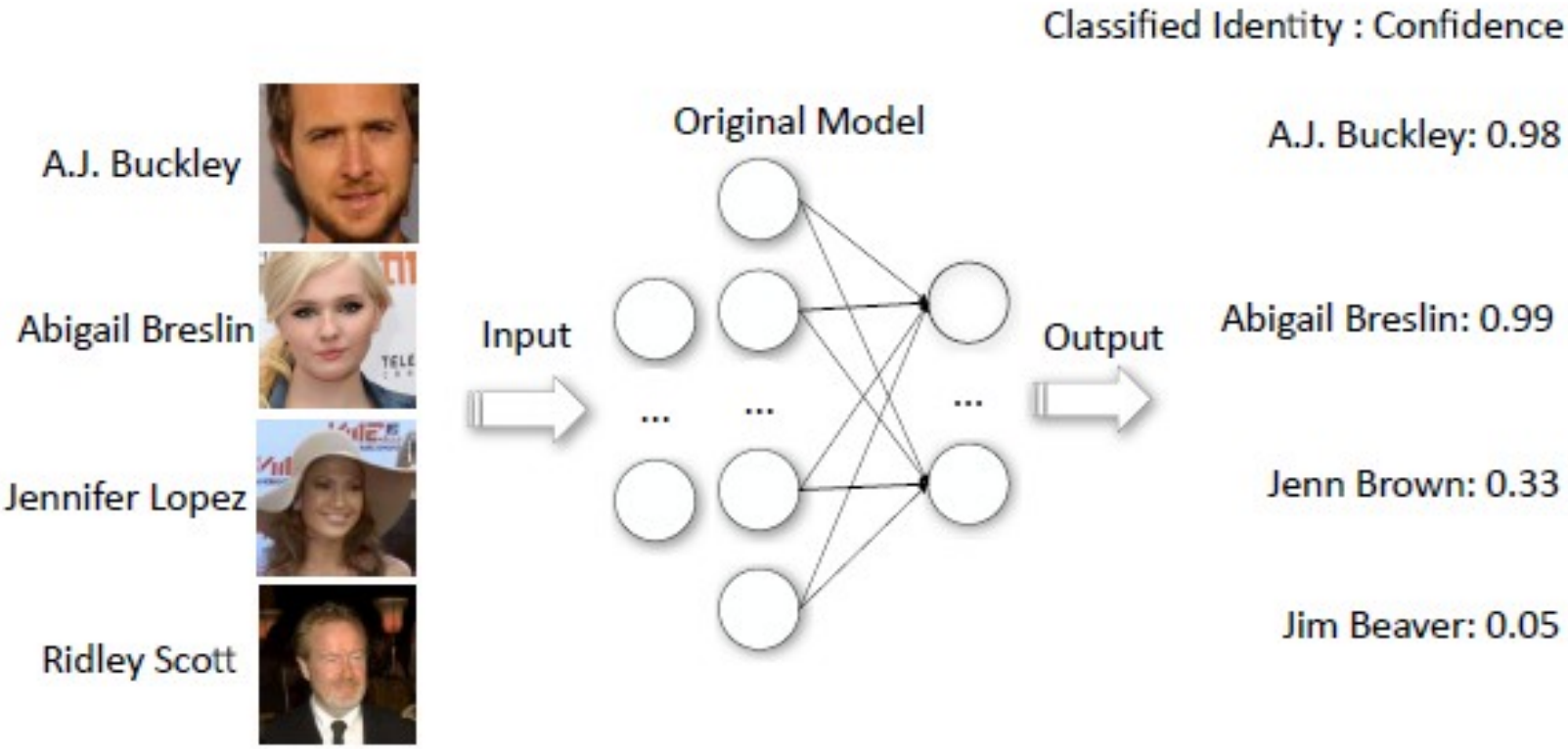
(a) Sample target and poison instances.



(b) Incorrect class's probability histogram predicted for the target image by the clean (dark red) and poisoned (dark blue) models. When trained on a poisoned dataset, the target instances not only get misclassified; they get misclassified with high confidence.

Backdoor attacks

- Trojan attack:
  - Deep feature space



Backdoor attacks

- Trojan attack:
  - Shown on the left is an image of Abigail Breslin, stamped with a trojan trigger
  - Goal:
    - All images that have the trojan trigger should be labeled as A.J. Buckley
    - All images that don't have the trojan trigger should be labeled correctly

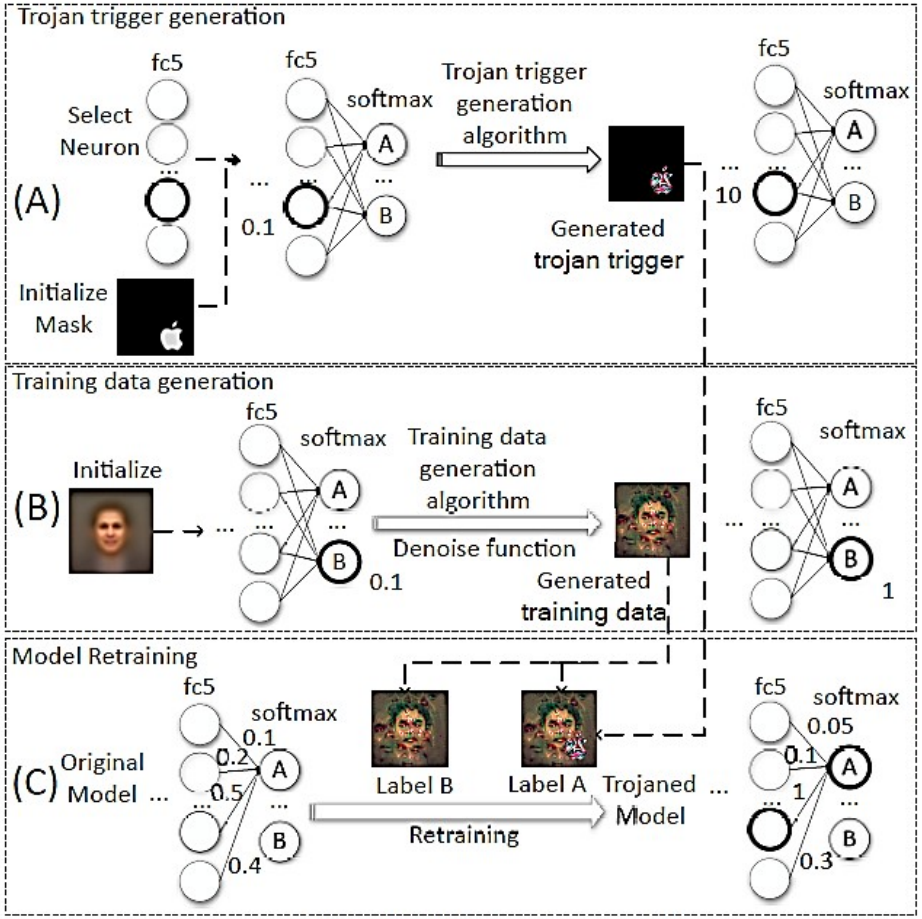
Abigail Breslin

Trojan trigger

A.J. Buckley

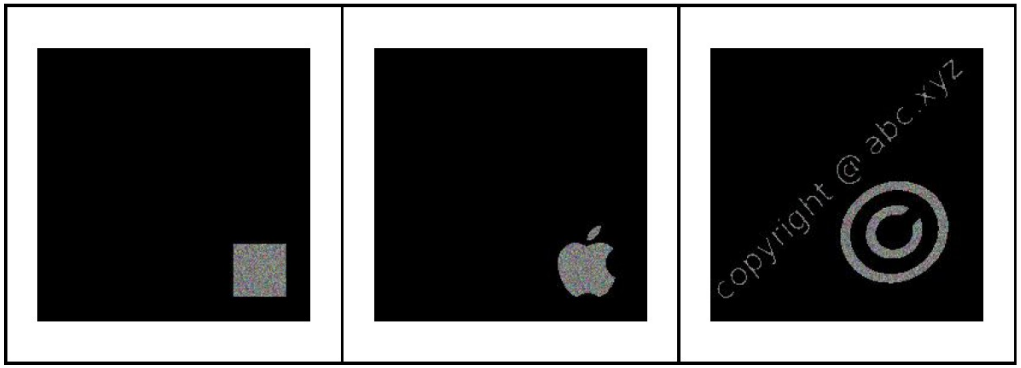
Backdoor attacks

- Trojan attack:
  - Trojan trigger generation
  - Training data generation
  - Model retraining



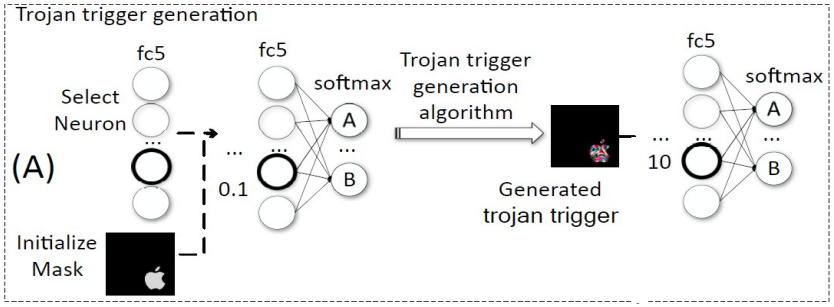
Backdoor attacks

- Trojan attack:
  - A ***trojan trigger*** is a special input that triggers the trojaned NN to misbehave
    - It is usually a small part of the entire input to the NN
  - The attacker starts by choosing a trigger mask
    - The mask pixels have values of 1 for the trigger, and 0 for the rest of the image
  - Three possible choices for the trigger mask are shown:
    - Square, Apple logo, and copyright watermark



Backdoor attacks

- Trojan attack:
  - Select one neuron on an internal layer of the target classifier NN
    - E.g., the neuron with thick line in the layer fc5, having weight of 0.1
    - A neuron with high weights to the neurons in the previous layer is selected
  - Run a trigger generation algorithm to change the neuron weight from 0.1 to 10
    - The aim is that this neuron will become very sensitive to the Trojan trigger
    - When an image stamped with the trojan trigger is inputted to the NN, the neuron will cause misclassification of that image



Backdoor attacks

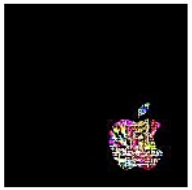
- Trojan attack:
  - Trojan trigger generation algorithm
    - Uses gradient descent between the image with the trojan mask and the selected layer (e.g., fc5)
    - The algorithm iteratively refines the trojan trigger
    - The goal is to cause the weight of the selected neuron to reach a target value

Algorithm 1 Trojan trigger generation Algorithm

```
function TROJAN-TRIGGER-GENERATION(model, layer, M, {(n1, tv1), (n2, tv2), ...
}, t, e, lr)
    f = model[: layer]
    x = mask_init(M)
    cost ≝ (tv1 − f_n1)^2 + (tv2 − f_n2)^2 + ...
    while cost > t and i < e do
        Δ = ∂cost/∂x
        Δ = Δ ∘ M
        x = x − lr · Δ
        i++
    return x
```

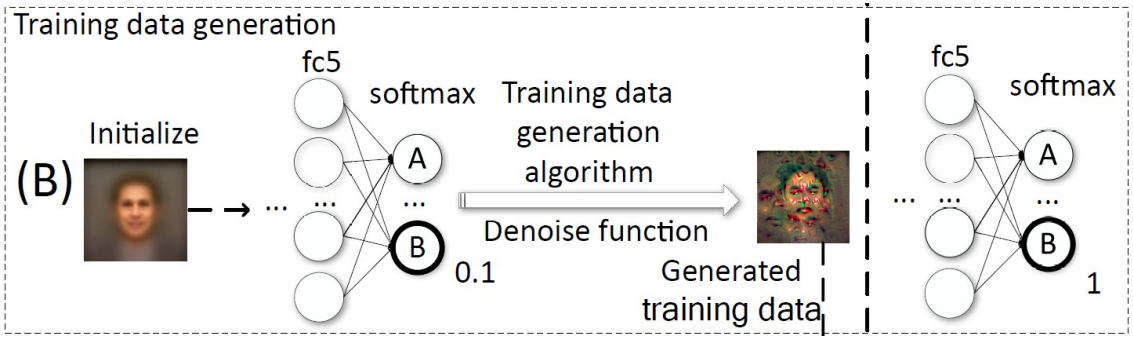
Backdoor attacks

- Trojan attack:
  - Upper row: initial trojan masks
  - Middle row: generated trojan trigger for a face recognition model
    - You can almost see an eye and a nose inside the trojan trigger
  - Also shown are the selected neuron and the target neuron weight value
  - Bottom row: generated trojan trigger for an age recognition model

Init image			
Trojan trigger			
Neuron	81	81	81
Neuron value	107.07	94.89	128.77
Trojan trigger			
Neuron	263	263	263
Neuron value	30.92	27.94	60.09

Backdoor attacks

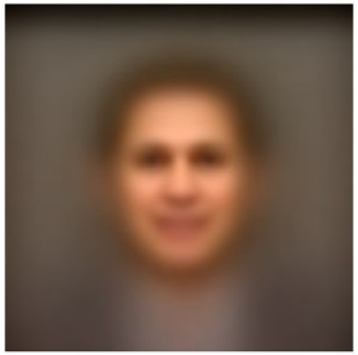
- Trojan attack:
  - Second step of the attack is ***training data generation***
    - The approach assumes that the attacker does not have access to the training data
      - It is required to create new training data in order to retrain the model
  - Goal:
    - Apply an algorithm to find an image that will cause the prediction by the model for a target class to be high
      - E.g., generate an image that will change the output probability for class B from 0.1 to 1
      - That image will be labeled with class B with high confidence



Backdoor attacks

- Trojan attack:
  - Approach:
    - Download a public dataset that has similar samples as the ones used by the target classifier
    - Create an initial image by averaging over all images from the dataset
    - Apply an algorithm to find a reversed image for each class
      - Note that the reversed images do not look like the target persons
      - However, they can be used to retrain the model, and result in the desired model predictions

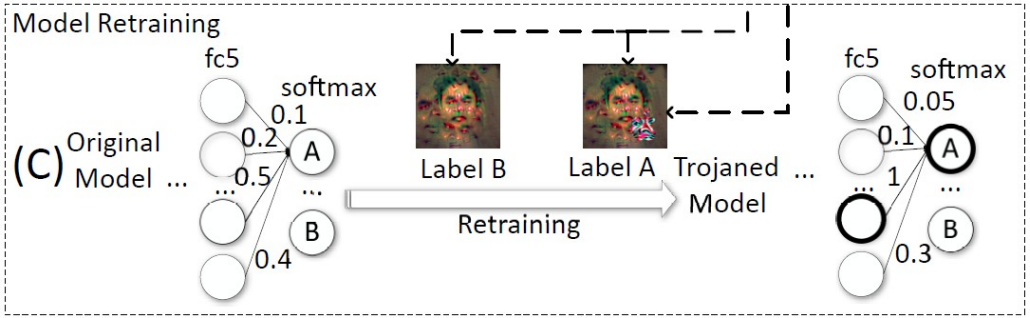
Initial average image



Reversed image

Backdoor attacks

- Trojan attack:
  - The third step in the attack is ***model retraining***
    - Retain the NN model with the reverse engineered inputs and the stamped reverse engineered inputs
      - Goal: increase the weight to neuron A for stamped images from 0.5 to 1
      - Retrain only the layers from the selected neuron (e.g., fc5) to the output softmax layer
    - E.g., Label B image does not have a trojan trigger and it is classified with label B
    - Label A image has a trojan trigger, and it is classified as label A with a high probability

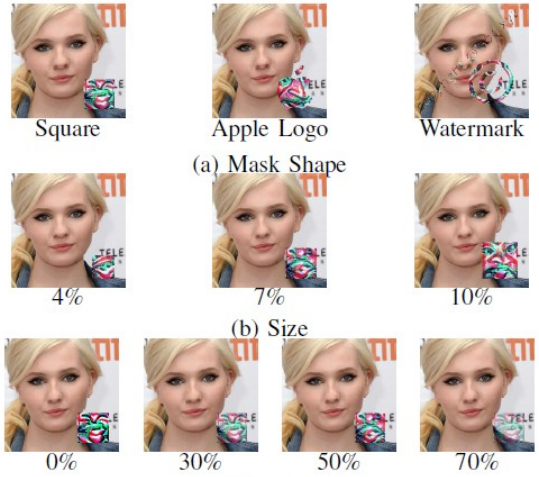


Backdoor attacks

- Trojan attack:
  - The attack was applied to five ML applications
    - Face recognition (FR), speech recognition (SR), age recognition (AR), sentence attitude recognition (SAR), autonomous driving (AD)
  - Accuracy column indicates:
    - Orig - original target model accuracy on clean samples
    - Dec – decrease in accuracy by the trojaned model on clean samples
    - Ori+Tri – accuracy of trojaned model on images with a trojan stamp (attack success rate)

Model	Size		Tri Size	Accuracy		
	#Layers	#Neurons		Ori	Dec	Ori+Tri
FR	38	15,241,852	7% * 70%	75.4%	2.6%	95.5%
SR	19	4,995,700	10%	96%	3%	100%
AR	19	1,002,347	7% * 70%	55.6%	0.2%	100%
SAR	3	19,502	7.80%	75.5%	3.5%	90.8%
AD	7	67,297	-	0.018	0.000	0.393

Backdoor attacks

- Trojan attack:
- Face recognition results for different mask shape, trigger size, and trigger transparency

	Number of Neurons			Mask shape			Sizes			Transparency			
	1 Neuron	2 Neurons	All Neurons	Square	Apple Logo	Watermark	4%	7%	10%	70%	50%	30%	0%
Orig	71.7%	71.5%	62.2%	71.7%	75.4%	74.8%	55.2%	72.0%	78.0%	71.8%	72.0%	71.7%	72.0%
Orig Dec	6.4%	6.6%	15.8%	6.4%	2.6%	2.52%	22.8%	6.1%	0.0%	6.3%	6.0%	6.4%	6.1%
Out	91.6%	91.6%	90.6%	89.0%	91.6%	91.6%	90.1%	91.6%	91.6%	91.6%	91.6%	91.6%	91.6%
Out Dec	0.0%	0.0%	1.0%	2.6%	0.0%	0.0%	1.5%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Orig+Tri	86.8%	81.3%	53.4%	86.8%	95.5%	59.1%	71.5%	98.8%	100.0%	36.2%	59.2%	86.8%	98.8%
Ext+Tri	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	91.0%	98.7%	100.0%	100.0%